

ภาควิชาคณิตศาสตร์ คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เอกสารประกอบการสอน

040223101 การเขียนโปรแกรมคอมพิวเตอร์สำหรับคณิตศาสตร์ 1

(Computer Programming for Mathematics 1)

บทที่ 10

การคำนวณเชิงตัวเลขด้วย NumPy

(Numerical Computing (NumPy))

ผู้สอน: รศ.ดร.อนุชิต จิตพัฒนกุล

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อศึกษาบทนี้จบ นักศึกษาสามารถ

- อธิบายข้อดีของอาร์เรย์ NumPy เทียบกับลิสต์ของ Python สำหรับงานคำนวณได้
- สร้างอาร์เรย์หนึ่งมิติและสองมิติ (เมทริกซ์) ด้วย NumPy ได้
- ทำการคำนวณแบบเวกเตอร์ (vectorized) กับอาร์เรย์ได้
- ดำเนินการกับเมทริกซ์ เช่น การบวก การคูณ และทรานสโพสได้
- ประยุกต์ NumPy แก้ปัญหาทางคณิตศาสตร์ เช่น ระบบสมการเชิงเส้นได้

CLO: เนื้อหาบทนี้สอดคล้องกับผลลัพธ์การเรียนรู้ของรายวิชา (CLO 5)

บทที่ 10 การคำนวณเชิงตัวเลขด้วย NumPy

การคำนวณทางคณิตศาสตร์และวิทยาศาสตร์มักเกี่ยวข้องกับชุดข้อมูลขนาดใหญ่ เวกเตอร์ และเมทริกซ์ NumPy (Numerical Python) เป็นไลบรารีที่ออกแบบมาเพื่อทำงานเหล่านี้โดยเฉพาะ ให้โครงสร้างข้อมูล อาร์เรย์ที่คำนวณได้เร็วและเขียนกระชับกว่าการใช้ลูปกับลิสต์ บทนี้เป็นบทสุดท้ายที่บูรณาการความรู้ ทั้งหมดสู่การคำนวณเชิงตัวเลข

10.1 รู้จัก NumPy และการนำเข้า

NumPy เป็นไลบรารีภายนอกที่ต้องติดตั้งก่อน (โดยทั่วไปติดตั้งด้วยคำสั่ง `pip install numpy` หรือมาพร้อม Anaconda) นิยมนำเข้าด้วยชื่อย่อ `np` โครงสร้างหลักคือ อาร์เรย์ (ndarray) ซึ่งเก็บข้อมูล ชนิดเดียวกันและคำนวณทั้งก้อนได้ในคราวเดียว

ไฟล์ 10array1.py

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
print(a)
print(type(a))
print("จำนวนสมาชิก:", a.size)
```

ผลลัพธ์:

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
จำนวนสมาชิก: 5
```

10.2 การสร้างอาร์เรย์

คำสั่ง	ความหมาย	ตัวอย่างผลลัพธ์
<code>np.array([1,2,3])</code>	สร้างจากลิสต์	<code>[1 2 3]</code>
<code>np.zeros(3)</code>	อาร์เรย์ศูนย์ 3 ตัว	<code>[0. 0. 0.]</code>
<code>np.ones(3)</code>	อาร์เรย์หนึ่ง 3 ตัว	<code>[1. 1. 1.]</code>
<code>np.arange(0,10,2)</code>	ลำดับเลขแบบ range	<code>[0 2 4 6 8]</code>
<code>np.linspace(0,1,5)</code>	5 ค่าเท่าระยะใน [0,1]	<code>[0. 0.25 0.5 0.75 1.]</code>

ไฟล์ 10create.py

```
import numpy as np
print(np.zeros(4))
print(np.arange(1, 6))
print(np.linspace(0, 10, 5))
```

ผลลัพธ์:

```
[0. 0. 0. 0.]
[1 2 3 4 5]
[ 0.   2.5  5.   7.5 10. ]
```

10.3 การคำนวณแบบเวกเตอร์ (Vectorized)

จุดเด่นของ NumPy คือการคำนวณกับทุกสมาชิกพร้อมกันโดยไม่ต้องเขียนลูป เรียกว่า **vectorization** ซึ่งเขียนสั้นและทำงานเร็ว ตัวดำเนินการทางคณิตศาสตร์จะกระทำกับสมาชิกทุกตัว

ไฟล์ 10vector.py

```
import numpy as np
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
print(a + b)
print(a * 2)
print(a ** 2)
print(np.sqrt(a))
```

ผลลัพธ์:

```
[11 22 33 44]
[2 4 6 8]
[ 1  4  9 16]
[1.  1.41421356 1.73205081 2.  ]
```

NumPy มีฟังก์ชันทางสถิติสำเร็จรูปที่ใช้กับอาร์เรย์ได้ทันที

ไฟล์ 10stat.py

```
import numpy as np
data = np.array([4, 8, 15, 16, 23, 42])
print("ผลรวม:", data.sum())
print("เฉลี่ย:", data.mean())
print("ส่วนเบี่ยงเบนมาตรฐาน:", round(data.std(), 4))
print("สูงสุด/ต่ำสุด:", data.max(), data.min())
```

ผลลัพธ์:

```
ผลรวม: 108
เฉลี่ย: 18.0
ส่วนเบี่ยงเบนมาตรฐาน: 12.3153
สูงสุด/ต่ำสุด: 42 4
```

10.4 อาร์เรย์สองมิติและเมทริกซ์

อาร์เรย์สองมิติใช้แทนเมทริกซ์ สร้างจากลิสต์ของลิสต์ คุณสมบัติ shape บอกขนาด (จำนวนแถว, จำนวน หลัก) การเข้าถึงสมาชิกใช้ [แถว, หลัก]

ไฟล์ 10matrix1.py

```
import numpy as np
M = np.array([[1, 2, 3],
              [4, 5, 6]])
print(M)
print("ขนาด:", M.shape)
print("สมาชิกแถว 0 หลัก 2:", M[0, 2])
```

ผลลัพธ์:

```
[[1 2 3]
 [4 5 6]]
ขนาด: (2, 3)
สมาชิกแถว 0 หลัก 2: 3
```

10.5 การดำเนินการกับเมทริกซ์

NumPy รองรับการดำเนินการพีชคณิตเชิงเส้น การคูณเมทริกซ์ใช้ตัวดำเนินการ @ (ต่างจาก * ที่คูณทีละ สมาชิก) และทรานสโพสใช้ .T

ไฟล์ 10matrix2.py

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print("ผลบวก:\n", A + B)
print("ผลคูณเมทริกซ์:\n", A @ B)
print("ทรานสโพสของ A:\n", A.T)
```

ผลลัพธ์:

```
ผลบวก:
[[ 6  8]
 [10 12]]
ผลคูณเมทริกซ์:
[[19 22]
 [43 50]]
ทรานสโพสของ A:
[[1 3]
 [2 4]]
```

10.6 ตัวอย่างประยุกต์: การแก้ระบบสมการเชิงเส้น

NumPy แก้ระบบสมการเชิงเส้น $Ax = b$ ได้ด้วยฟังก์ชัน `np.linalg.solve` ตัวอย่างแก้ระบบสมการ $2x + y = 11$ และ

$$x + 3y = 18$$

ไฟล์ 10solve.py

```
import numpy as np
A = np.array([[2, 1], [1, 3]])
b = np.array([11, 18])
x = np.linalg.solve(A, b)
print(f"x = {x[0]:.1f}, y = {x[1]:.1f}")
```

ผลลัพธ์:

$x = 3.0, y = 5.0$

เราสามารถหาดีเทอร์มิแนนต์และเมทริกซ์ผกผันได้เช่นกัน

ไฟล์ 10linalg.py

```
import numpy as np
A = np.array([[2, 1], [1, 3]])
print("ดีเทอร์มิแนนต์:", np.linalg.det(A))
print("เมทริกซ์ผกผัน:\n", np.linalg.inv(A))
```

ผลลัพธ์:

ดีเทอร์มิแนนต์: 5.000000000000001

เมทริกซ์ผกผัน:

```
[[ 0.6 -0.2]
 [-0.2  0.4]]
```

10.7 สรุปท้ายบทและสรุปรายวิชา

NumPy ให้โครงสร้างอาร์เรย์ที่คำนวณได้รวดเร็วและกระชับสำหรับงานเชิงตัวเลข เราสร้างอาร์เรย์ด้วย `array`, `zeros`, `arange`, `linspace` คำนวณแบบเวกเตอร์โดยไม่ต้องใช้ลูป และจัดการเมทริกซ์ได้ครบถ้วน ทั้งการบวก คูณ ทรานสโพส และการแก้ระบบสมการเชิงเส้น บทนี้เป็นการบูรณาการความรู้ทั้งรายวิชา ตั้งแต่ ตัวแปร เงื่อนไข การทำซ้ำ โครงสร้างข้อมูล และฟังก์ชัน สู่การประยุกต์แก้ปัญหาทางคณิตศาสตร์อย่างเป็นระบบ ซึ่งเป็นรากฐานสำคัญสำหรับการศึกษาด้านการคำนวณและวิทยาการข้อมูลในระดับสูงต่อไป

แบบฝึกหัดท้ายบท

1. จงสร้างอาร์เรย์ของเลข 1 ถึง 10 แล้วแสดงผลรวม ค่าเฉลี่ย และอาร์เรย์ของกำลังสองของแต่ละตัว
2. จงสร้างอาร์เรย์สองชุดแล้วแสดงผลบวก ผลต่าง และผลคูณทีละสมาชิก
3. จงสร้างเมทริกซ์ขนาด 3×3 แล้วแสดงทรานสโพสและผลรวมของสมาชิกทั้งหมด
4. จงใช้ NumPy แก้ระบบสมการ $3x + 2y = 12$ และ $x - y = 1$
5. จงสร้างอาร์เรย์ 100 ค่าเท่าระยะในช่วง 0 ถึง 2π แล้วคำนวณค่า $\sin$ ของแต่ละค่า

เฉลยแบบฝึกหัดบทที่ 10

ข้อ 1

```
import numpy as np
a = np.arange(1, 11)
print("ผลรวม:", a.sum())
print("เฉลี่ย:", a.mean())
print("กำลังสอง:", a ** 2)
```

ผลลัพธ์:

```
ผลรวม: 55
เฉลี่ย: 5.5
กำลังสอง: [ 1  4  9 16 25 36 49 64 81 100]
```

ข้อ 4 แก่ระบบสมการ

```
import numpy as np
A = np.array([[3, 2], [1, -1]])
b = np.array([12, 1])
x = np.linalg.solve(A, b)
print(f"x = {x[0]:.1f}, y = {x[1]:.1f}")
```

ผลลัพธ์:

```
x = 2.8, y = 1.8
```

ข้อ 5 sin ของช่วง 0 ถึง 2π

```
import numpy as np
x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x)
print("ค่าแรก 3 ตัว:", y[:3])
print("ค่ามากที่สุด:", round(y.max(), 4))
```

ผลลัพธ์:

```
ค่าแรก 3 ตัว: [0.          0.06342392  0.12659245]
ค่ามากที่สุด: 0.9999
```